



Vue.js Props

정의 · 선언 · 타입/기본값 · 단방향 데이터 흐름 · 검증 · 불변성 · 실습

Agenda

Props 개념부터 실전까지

- 1 Props란? (부모→자식 데이터 전달)
- 2 선언 방법: 문자열 배열 vs 객체형
- 3 Props 실습
- 4 Props 실습 - 기본값 설정
- 5 Props 실습 - 배열 전달
- 6 불변성 및 주의사항
- 7 요약

Props란 무엇인가?

- 부모 컴포넌트가 자식에게 값을 전달하는 커스텀 속성
- 컴포넌트 간 결합도를 낮추며 재사용성 향상
- 단방향 데이터 흐름 원칙 준수 (One-Way Data Flow)
- 타입 검증, 기본값, 필수 여부 지정 가능



Props 실습: 부모 → 자식 데이터 전달

아래는 title 속성을 props로 전달하는 예시입니다.

Hello Props

Vue.js 재밌다!

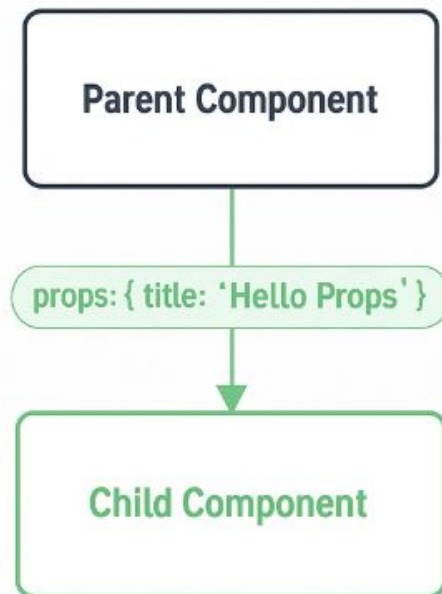
Props란 무엇인가?

```
<!-- ParentComponent.vue -->
<script setup>
import ChildComponent from './ChildComponent.vue'
</script>

<template>
  <main style="text-align:center;">
    <h1>💡 Props 실습: 부모에서 자식으로 데이터 전달</h1>
    <p>아래는 props를 통해 title 값을 전달한 예시입니다.</p>

    <!-- 자식 컴포넌트 호출 -->
    <ChildComponent title="Hello Props" />
  </main>
</template>
```

Props 데이터 흐름



```
<script setup>
  const props = defineProps( title:
    { type:String,required:true } )
</script>
<template>
  <section class=card>
    h2 {{ props.title }}/2>
  </section>
<style scoped>
  .card {
</style>
```

부모에서 자식으로의 데이터 전달 (One-Way Data Flow)

Props 선언: 문자열 배열 vs 객체형

문자열 배열 선언

간단하게 **props** 이름만 나열할 때 사용합니다. 타입 체크나 검증이 필요 없을 때 적합합니다.

```
// <script setup>
const props = defineProps(['title', 'likes'])

// Options API
export default {
  props: ['title', 'likes']
}
```

- 빠르게 프로토타이핑할 때 편리
- 타입 체크 없음

객체형 선언 (권장)

타입 체크, 기본값, 유효성 검증 등 상세 옵션을 제공합니다.

```
// <script setup>
const props = defineProps({
  title: String,
  likes: Number,
  isPublished: {
    type: Boolean,
    required: true,
    default: false
  }
})
```

- 자체 문서화 및 타입 검증
- 개발자 경험 향상 및 오류 방지

Props 실습

Props 선언 비교 데모

Hello Props

likes: 4

published: YES

Second Card

likes: 0

published: NO

Props 실습

```
<template>
  <main style="text-align:center; margin-top:1.5rem;">
    <h1>Props 선언 비교 데모</h1>

    <!-- 문자열/숫자/불리언 전달 -->
    <ChildComponent title="Hello Props" :likes="4" is-published />

    <!-- 바인딩으로 다른 값 전달 -->
    <ChildComponent title="Second Card" :likes="0" :is-published="false"
  </main>
</template>

<script setup>
import ChildComponent from './components/ChildComponent.vue'
</script>
```

Props 선언 비교 데모

Hello Props

likes: 4

published: YES

Second Card

likes: 0

published: NO

Props 실습

```
<template>
  <section class="card">
    <h2>{{ props.title }}</h2>
    <p>likes: {{ props.likes }}</p>
    <p>published: {{ props.isPublished ? 'YES' : 'NO' }}</p>
  </section>
</template>

<script setup>
const props = defineProps({
  title: { type: String, required: true },
  likes: { type: Number, default: 0 },
  isPublished: { type: Boolean, default: false }
})
</script>
```


Props - 자주 사용하는 타입

타입	예시	설명
 String	<code>"홍길동"</code>	텍스트 형식의 문자열 데이터
 Number	<code>30</code>	정수 또는 소수점 숫자 데이터
✓ Boolean	<code>true</code> , <code>false</code>	참/거짓 값을 나타내는 논리 데이터
 Array	<code>[1, 2, 3]</code>	순서가 있는 여러 값의 묶음
 Object	<code>{ name: '홍길동' }</code>	키-값 쌍으로 구성된 복합 데이터

Props - 기본값 설정 예시

```
<script setup>
const props = defineProps({
  title: { type: String, default: '제목 없음' },
  count: { type: Number, default: 0 },
  isActive: { type: Boolean, default: false }
})
</script>

<template>
  <p>{{ title }} ({{ count }})</p>
  <p>활성 상태: {{ isActive ? 'ON' : 'OFF' }}</p>
</template>
```

📌 설명

- 1 title이 안 넘어오면 "제목 없음"으로 표시
- 2 count가 없으면 0으로 표시
- 3 isActive가 없으면 false로 처리(OFF)

실행 결과 예시

제목 없음 (0)
활성 상태: OFF

💡 Tip: 객체나 배열의 경우 **default 값은 반드시 함수로** 반환해야 합니다. 예: default: () => []

Props - 기본값 설정 예시

```
<template>
  <main style="text-align:center; margin-top:1.5rem;">
    <h1>Props 선언 비교 데모</h1>

    <MyCard title="Vue 강의" :count="3" is-active />
    <MyCard :count="0" :is-active="false" />
  </main>
</template>

<script setup>
import MyCard from './components/MyCard.vue'
</script>
```

Props 선언 비교 데모

Vue 강의

좋아요 수: 3

활성 상태: ON

기본 제목

좋아요 수: 0

활성 상태: OFF

배열(Array)을 props로 전달하기

Vue.js에서 배열 Props를 안전하게 전달하는 방법

학습 목표



배열을 props로 전달하는 기본 원리 이해



콜론(:) 유무에 따른 문자열 vs 배열 차이 구분



배열 기본값을 함수로 반환하는 패턴 숙지

배열(Array)을 props로 전달하기

핵심 개념



배열은 여러 데이터를 묶어서 전달

컴포넌트 간 다수의 관련 데이터를 효율적으로 전달할 때 사용합니다.



콜론(:)의 역할

콜론 없이 사용: `items=['사과', '배']` → 문자열로 전달

콜론 사용: `:items=['사과', '배']` → JavaScript 배열로 전달



기본값은 함수로 반환

올바른 방법: `default: () => []` (팩토리 함수)

잘못된 방법: `default: []` (모든 인스턴스에서 같은 배열 참조)



주의 사항:

배열과 객체는 참조형이므로 여러 컴포넌트에서 공유될 수 있습니다.

각 인스턴스마다 고유한 배열을 갖게 하려면 반드시 함수를 사용해 새 배열을 반환하세요.

배열(Array)을 props로 전달하기

자식 컴포넌트: ChildList.vue

```
<template>
  <div class="card">
    <h3>{{ title }}</h3>
    <ul>
      <li v-for="(item, idx) in items" :key="idx">
        {{ idx + 1 }}. {{ item }}
      </li>
    </ul>
  </div>
</template>

<script setup>
  const props = defineProps({
    title: { type: String, default: '항목 리스트' },
    items: { type: Array, default: () => [] },
  })
</script>
```

배열(Array)을 props로 전달하기

부모 컴포넌트: App.vue

```
<template>
  <main style="text-align:center;">
    <h2>배열 Props 전달 예시</h2>

    <!-- (1) 문자열로 전달 - 배열처럼 보이지만 실제로는 문자열 -->
    <ChildList title="문자열 props" items="['사과', '배', '포도']" />

    <!-- (2) 콜론(:)을 붙여 실제 배열로 전달 -->
    <ChildList title="배열 props" :items="['사과', '배', '포도']" />

    <!-- (3) 변수로 전달 -->
    <ChildList title="변수 전달" :items="fruits" />
  </main>
</template>

<script setup>
import ChildList from './components/ChildList.vue'

const fruits = ['복숭아', '수박', '자두']
</script>
```

배열(Array)을 props로 전달하기

설명 포인트 - 배열 Props 전달 시 주의사항

1 콜론(:)은 JavaScript 표현식

`:items=["'사과','배','포도']"`

- 실제 배열 객체로 평가되어 전달됩니다
- 자식 컴포넌트에서 `v-for` 로 순회 가능

2 콜론이 없으면 문자열

`items=["'사과','배','포도']"`

- 문자열 그대로 전달됩니다 `"['사과','배','포도']"`
- `props.items.length` 는 배열 길이가 아니라 문자열 길이가 반환됩니다 😓

문자열 props

- 1. [
2. '
3. 사
4. 과
5. '
6. ,
7.
8. '
9. 배
10. '
11. ,
12.
13. '
14. 포
15. 도
16. '
17.]

배열 props

- 1. 사과
2. 배
3. 포도

변수 전달

- 1. 복숭아
2. 수박
3. 자두

Props 불변성과 주의사항

- 직접 변경 금지: 컴포넌트에서 자신의 props를 직접 수정해서는 안 됩니다

Vue 경고:

Avoid mutating a prop directly since the value will be overwritten whenever the parent component re-renders.

- 객체/배열 참조 문제: 객체와 배열은 참조로 전달되므로 내부 속성을 변경하면 부모 상태에 영향을 줍니다

```
<script setup>
```

```
const props = defineProps({  
  items: { type: Array, default: () => [] }  
})
```

```
// 1) props의 스냅샷(복사본)으로 로컬 상태 유지  
const localItems = ref([...props.items])
```

```
// 2) 로컬을 수정  
function add() {  
  localItems.value = [...localItems.value, '추가']  
}  
</script>
```

호출 전

['사과', '배', '포도']

호출 후

['사과', '배', '포도', '추가']

요약

- 1 단방향 데이터 흐름 - Props는 항상 부모→자식 단방향 데이터 채널로 사용됩니다.
- 2 객체형 선언 사용 - 문자열 배열보다 타입/기본값/검증을 포함한 객체형 선언이 유지보수성을 높입니다.
- 3 불변성 유지 - Props를 직접 변경하지 않습니다.

참고 : <https://v3-docs.vuejs-korea.org/guide/components/props.html>